

CLAUDE CODE DEEP DIVE WORKSHOP

Chapter 4 - Settings

settings, Permissions, Hooks, MCP, Commands 완전 정복

Update : 2026.07 / Claude Code v2.1 최신 채널 기준, 공식 문서 code.claude.com/docs 반영

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com

Chapter 4 Agenda

9 Parts / 115 Slides

PART 01	Settings 체계	스코프, 우선순위, 키 카탈로그, 리로드 / 14p
PART 02	Permissions	규칙 문법, 모드 6종, 설계 전략 / 15p
PART 03	Hooks 아키텍처	30 이벤트, 매처, 입출력, 5 핸들러 / 14p
PART 04	Hooks 실전	HTTP, MCP tool, prompt 효과 레시피 / 11p
PART 05	MCP 구성	전송, 스코프, OAuth, 도구 검색 / 13p
PART 06	MCP 운영과 보안	카탈로그, 신뢰 모델, 통제 / 10p
PART 07	Commands와 Skills	커스텀 명령, SKILL.md, 선택 가이드 / 13p
PART 08	통합과 트러블슈팅	풀스택 구성, 진단 4도구 / 8p
PART 09	Recap & Labs	요약, FAQ, 실습 4종 / 12p

Chapter 4 Learning Objectives

본 챕터에서 달성할 학습 목표

AT THE END OF THIS CHAPTER

설정 스코프와 우선순위를 설계하고, 권한 규칙과 모드를 조율하며, 혹은 수명주기를 자동화하고, MCP와 커맨드로 나만의 개발 플랫폼을 완성할 수 있습니다.

OBJ 1

설정 설계

4스코프와 병합 규칙으로
개인, 팀 설정을 구조화.

OBJ 2

권한 조율

규칙 문법과 모드 6종으로
마찰 없는 안전선 구축.

OBJ 3

혹 자동화

이벤트와 핸들러 조합으로
워크플로 자동화.

OBJ 4

확장 통합

MCP, 커맨드, 스킬로 팀 표
준 플랫폼 완성.

9 Parts

구조화된 학습 경로

4 Labs

Chapter 4 실습 랩

30 Events

혹 이벤트 카탈로그

~3.5h + 1.5h

강의와 실습 시간

.claude 디렉토리 지도

오늘 다룰 모든 것이 사는 곳

settings.json (.local)

설정, 권한, 혹은, env

Part 1, 2, 3

CLAUDE.md, rules/

프로젝트 지침과 규칙

Chapter 1 복습

agents/

서브에이전트 정의

Chapter 2

commands/, skills/

커스텀 명령과 스킬

Part 7

hooks/ (스크립트)

혹 핸들러 스크립트 관례 위치

Part 3, 4

.mcp.json (루트)

프로젝트 MCP 서버

Part 5, 6

~/claude

사용자 스코프 쌍둥이

커밋 가능

팀 자산화 범위

local 제외

gitignore 대상

한 지도

챕터 4의 범위

04

Settings

설정, 권한, 혹은 MCP, 커맨드의 정밀 조율

CHAPTER 04 / PART 01

Settings 체계

스코프와 우선순위, 그리고 살아있는 리로드

- 4스코프와 우선순위 5단
- 파일 지도와 \$schema 자동완성
- 주요 키 카탈로그 3분류
- 라이브 리로드와 관용, 엄격 파싱

설정 스코프 4계층

어디에 두면 누구에게 적용되는가

Managed	서버 관리, plist/레지스트리, 시스템 파일	조직 전체, IT 배포 (Ch.3)
User	~/.claude/settings.json	나의 전 프로젝트, 비공유
Project	.claude/settings.json	저장소 협업자 전원, 커밋 공유
Local	.claude/settings.local.json	이 저장소의 나만, gitignore
용도 구분	취향은 User, 팀 표준은 Project	실험과 개인 예외는 Local

우선순위 5단

같은 키가 충돌하면 누가 이기는가

1 Managed

무엇으로도 재정의 불가 (Ch.3의 강제 계층)

2 CLI 인자

실행 시 플래그, 그 세션 한정 임시 재정의

3 Local

settings.local.json, 프로젝트와 사용자 값을 덮음

4 Project

팀 공유 표준, 사용자 값을 덮음

5 User

아무도 지정하지 않았을 때의 내 기본값

기능별 파일 지도

무엇이 어느 파일에 저장되는가

Settings	~/ .claude/settings.json, .claude/settings(.local).json	본 파트
MCP 서버	User/Local은 ~/ .claude.json, Project는 .mcp.json	Part 5
Subagents	~/ .claude/agents/, .claude/agents/	Chapter 2
CLAUDE.md	~/ .claude/, 프로젝트 루트, CLAUDE.local.md	Chapter 1
~/ .claude.json	OAuth 세션, 프로젝트별 상태, 캐시	직접 편집 지양
백업	설정 파일 자동 백업 최근 5개 보관	복구 안전망

settings.json 해부

\$schema로 자동완성까지

.claude/settings.json

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": ["Bash(npm run lint)", "Bash(npm run test *)"],
    "deny": ["Read(./env)", "Read(./secrets/**)"]
  },
  "env": {
    "CLAUDE_CODE_ENABLE_TELEMETRY": "1"
  },
  "companyAnnouncements": ["코드 리뷰 필수, 가이드는 wiki 참조"]
}
```

\$schema

에디터 자동완성

permissions

Part 2 주제

env

환경변수 배포

announcements

시작 배너

env 블록

설정 파일로 환경변수를 배포한다

ENV VIA SETTINGS

셀 프로필 대신 settings의 env 블록으로 환경변수를 스코프별로 배포합니다.

프로젝트에 커밋하면 팀 전체의 변수가 통일됩니다.

SCOPE

스코프 상속

User는 전역, Project는 팀,
Local은 나만.

USE 1

관측 배선

OTel 변수들을 프로젝트에 커밋
해 팀 계측.

USE 2

공급자 고정

USE_BEDROCK, 리전을
managed env로 강제 (Ch.3).

CARE

비밀 금지

키, 토큰은 env 블록이 아닌 볼
트 헬퍼로.

셀 대체

profile.d 불필요

커밋 배포

팀 통일 수단

No secrets

env 블록 원칙

env-vars 문서

전체 변수 목록

키 카탈로그 1 / 모델과 사고

지능을 조절하는 키들

<code>alwaysThinkingEnabled</code>	확장 사고 기본 활성화	/config로 토글
<code>availableModels</code>	선택 가능 모델 제한	메인, 서브에이전트, 스킬 공통
<code>enforceAvailableModels</code>	Default 선택지까지 강제	managed 짝 키
<code>advisorModel</code>	어드바이저 도구 모델	/advisor 실행 시 기록
<code>agent</code>	세션 기본 에이전트	Chapter 2의 --agent 설정판
<code>model</code> 주의	세션 시작 시 1회 읽음	중간 변경은 /model

키 카탈로그 2 / 운영

일상 운영을 조절하는 키들

<code>autoUpdatesChannel</code>	latest 기본, stable은 약 1주 지연	플릿은 stable 권장 (Ch.3)
<code>autoCompactEnabled</code>	컨텍스트 자동 압축	기본 true
<code>autoMemoryEnabled</code>	자동 메모리 축적	/memory로 토글
<code>cleanupPeriodDays</code>	세션 파일 정리 주기, 기본 30	고아 worktree도 정리
<code>attribution</code>	커밋, PR 서명 문구 커스텀	commit, pr 필드
<code>companyAnnouncements</code>	시작 공지 순환 표시	팀 공지판

키 카탈로그 3 / 끄기 스위치

disable 패밀리

<code>disableAllHooks</code>	전체 훅과 커스텀 상태줄 정지	managed 계층 존중
<code>disableBundledSkills</code>	번들 스킬, 워크플로 제거	자체 표준만 쓸 때
<code>disableAutoMode</code>	auto 모드 진입 봉쇄	"disable" 값
<code>disableAgentView</code>	백그라운드 에이전트 뷰 차단	고통제 환경
<code>disableRemoteControl</code>	원격 제어 기능 차단	MDM 배포 대상
<code>disableSideloadFlags</code>	--agents 등 우회 플래그 거부	managed 전용 (Ch.3)

라이브 리로드

저장하면 대부분 즉시 반영

LIVE RELOAD

설정 파일은 감시되어 저장 즉시 세션에 반영됩니다.

permissions, hooks, apiKeyHelper가 대표이고, 변경마다 ConfigChange 혹은 발화합니다.

LIVE

즉시 반영

permissions, hooks, 자격 헬퍼 등 대부분 키.

EVENT

ConfigChange

소스별 매치로 변경 감지 자동화 가능 (Part 3).

EXC 1

model

시작 시 1회, 세션 중엔 /model 명령.

EXC 2

outputStyle

시스템 프롬프트 일부, /clear나 재시작에 재구성.

파일 위치

반영 메커니즘

전 스코프

managed 포함 감시

2 예외

model, outputStyle

무재시작

일상 편집 경험

managed-settings.d 드롭인

팀별 정책 조각의 무충돌 병합

●●● /etc/claude-code/ (drop-in)

```
managed-settings.json      # 기본판 (먼저 병합)
managed-settings.d/
  10-telemetry.json        # 관측팀 조각
  20-security.json         # 보안팀 조각
  30-mcp-allowlist.json    # 플랫폼팀 조각

# 병합 규칙 (systemd 관례)
# 알파벳순 적용, 숫자 접두로 순서 통제
# 스칼라: 나중 파일이 덮음
# 배열: 이어붙임 + 중복 제거
# 객체: 깊은 병합 / 숨김 파일(.* )은 무시
```

.d 디렉토리
조각 배포 방식

10-, 20-
순서 접두 관례

팀 독립
단일 파일 조정 불필요

managed-mcp.json
동거 파일

관용 파싱 vs 엄격 파싱

오타 하나가 정책 전체를 죽이지 않게

PARSING SEMANTICS

managed 설정은 관용적으로, 개인 설정은 엄격하게 해석됩니다.

검증 오류는 시작 대화상자와 doctor 명령이 알려줍니다.

Managed (관용)

무효 항목만 제거, 나머지 강제 유지

보안 키는 fail-closed: 무효 allowlist는 빈 목록

minimumVersion류만 fail-open (기동 보장)

배포 전 claude doctor로 검증

User, Project, Local (엄격)

검증 실패 시 파일 전체 거부

오류는 시작 시 보고

\$schema로 사전 예방

고치기 전까지 해당 스코프 무시

/config

설정의 대화형 창구

~/proj \$ claude

> /config

탭형 설정 UI: 상태 확인과 옵션 토글

> /config verbose=true

UI 없이 단일 키 즉시 변경 (v2.1.181+)

자주 만지는 항목

Auto-compact, Session recap, Push 알림,

Thinking 기본값, 테마

어디에 저장되는지 표시되므로 스코프 확인 겸용

탭 UI
대화형 창구

key=value
원샷 변경

스코프 표시
저장 위치 확인

파일 병행
직접 편집도 유효

Part 1 Recap

Settings 체계 핵심 정리

KEY TAKEAWAY

스코프 4개, 우선순위 5단, 권한은 병합. 저장 즉시 반영되고, managed는 관용, 개인은 엄격.

이 문법 위에 나머지 여덟 파트가 올라갑니다.

핵심 정리

Managed > CLI > Local > Project > User

MCP만 저장 파일이 다름

\$schema로 자동완성

리로드 예외: model, outputStyle

드롭인 .d로 정책 조각 병합

다음 파트

Part 2 Permissions

규칙 문법 Tool(specifier)

모드 6종과 Shift+Tab

auto 분류기 커스텀

설계 전략과 실수

CHAPTER 04 / PART 02

Permissions

마찰 없는 안전선의 문법

- allow, ask, deny와 평가 순서
- Bash 패턴과 경로 그룹
- 모드 6종과 auto 분류기 커스텀
- 설계 전략과 흔한 실수

3동사와 평가 순서

deny가 항상 이긴다

THREE VERBS

규칙은 allow, ask, deny 세 목록에 삽니다.

도구 호출은 deny 대조가 먼저이고, allow에 있으면 조용히 실행, 어느 쪽도 아니면 모드가 기본 거동을 정합니다.

DENY

무조건 차단

일치 시 즉시 거부, 다른 목록보다 항상 우선.

ALLOW

무확인 실행

일치 시 프롬프트 없이 실행, 생산성의 축.

ASK

명시 확인

일치 시 반드시 확인 프롬프트, 회색 지대용.

ELSE

모드 위임

무일치 시 현재 권한 모드의 기본 거동.

deny 우선

평가 원칙

전 스코프 병합

목록 합집합

구체 우선

동렬 충돌 해석

모드 폴백

무일치 처리

규칙 문법 해부

Tool(specifier) 한 형식

rule anatomy

```
"Bash(npm run lint)"      # 도구(지정자)
```

```
# 도구만: 그 도구의 모든 호출
```

```
"WebFetch"                # 웹 페치 전부
```

```
# 지정자: 도구별 의미가 다름
```

```
# Bash -> 명령 패턴
```

```
# Read, Edit -> 파일 경로 패턴
```

```
# Agent -> 서버에이전트 타입 (Ch.2)
```

```
# mcp__server(__tool) -> MCP 도구 (Part 5)
```

1 형식

Tool(specifier)

도구별 의미

지정자 해석

Task 별칭

Agent 구표기 유효

혹 if 공용

같은 문법 재사용

Bash 패턴

정확 일치와 그룹

bash patterns

```
"Bash(npm run lint)"    # 정확히 이 명령만
"Bash(npm run test *)"   # 이 접두 + 임의 인자
"Bash(git *)"            # git 하위 전부

# deny 예시
"Bash(curl *)"           # 임의 curl 차단
"Bash(rm -rf *)"         #

# 주의: 셸 체이닝, 치환의 우회 가능성
# 권한은 1차 방어, 강한 봉쇄는 sandbox (Ch.3)
# 혹은 if의 서브커맨드 검사와 층이 다름 (Part 3)
```

정확/그룹

두 매칭 형태

* 인자

접두 허용 관용구

1차 방어

권한의 위상

sandbox

강한 봉쇄 계층

파일 경로 패턴

Read, Edit의 gitignore 스타일 그룹

path patterns

```
"Read(./env)"          # 프로젝트 루트의 .env
"Read(./env*)"          # .env.local 등 변형
"Read(./secrets/**)"    # 하위 전체
"Edit(*.ts)"            # 확장자 단위
"Read(~/.zshrc)"        # 홈 기준 경로
"Read(/etc/passwd)"     # 절대 경로는 // 접두

# ./ 프로젝트 루트 기준
# ** 깊이 무제한, * 한 단계
```

gitignore식
그룹 문법

./ 루트
상대 기준

// 절대
특수 접두

~/ 홈
홈 기준

특수 지정자

도구별 확장 규칙

Agent(Explore)	특정 서브에이전트 타입 통제	Chapter 2 복습
Agent	지정자 없이: 위임 자체를 통제	deny 시 전 위임 봉쇄
mcp__github	서버 전체 도구	Part 5, 6
mcp__github__get_issue	서버의 특정 도구	세밀 통제
WebFetch	도구 전체 (지정자 없이)	sandbox와 병용 (Ch.3)
additionalDirectories	규칙이 아닌 접근 범위 확장	다음 파트 상세

권한 모드 6종

Shift+Tab으로 순환

default

표준: 위험 작업마다 확인

일상 기본

acceptEdits

작업 경로 편집 자동 수락

반복 수정 세션

plan

읽기 전용 탐색, 계획 산출

설계 단계

auto

분류기가 명령을 심사 후 자동 결정

신뢰 저장소

dontAsk

확인 프롬프트 자동 거부

무인, 명시 allow만

bypassPermissions

전 확인 생략

격리 환경 한정, 조직 차단 대상

auto 모드 커스텀

분류기 규칙을 설정으로

settings.json (autoMode)

```
{ "autoMode": {  
  "environment": ["사내 모노레포, k8s 스테이징 접근 가능"],  
  "allow":      ["$defaults", "kubectl get, describe 계열"],  
  "soft_deny": ["$defaults", "terraform apply 금지"],  
  "hard_deny": ["IAM 정책 변경 시도"],  
  "classifyAllShell": true  
} }
```

산문 규칙 배열, \$defaults로 내장 규칙 상속 위치 지정

classifyAllShell: 모든 셸 명령을 분류기로 (v2.1.193+)

공유 프로젝트 설정에서는 미읽음, user/managed로

산문 규칙

자연어 배열

\$defaults

내장 상속 마커

3단 강도

allow, soft, hard

user/managed

유효 스코프

dontAsk 무인 운용

물을 수 없는 곳의 규칙

DONTASK MODE

확인이 필요한 호출을 자동 거부합니다.

명시 allow만 실행되므로, 무인 스크립트에서 허용 목록이 곧 능력의 전부가 됩니다.

SEMANTIC

거부 기본

프롬프트가 뜰 상황이면 그 호출은 거부.

PAIR

allow 설계

필요한 명령을 빠짐없이 명시 허용해야 동작.

WHERE

쓰는 곳

혹 게이트, cron, CI 등 응답자 없는 실행.

VS

bypass와 차이

생략이 아닌 거부, 안전 방향으로 넘어짐.

auto-deny

기본 거동

allowlist=능력

설계 관점

Ch.2 재회

scanner의 모드

fail-safe

bypass 대비

additionalDirectories

작업 범위를 넓히는 별도 키

settings.json (scope extension)

```
{ "permissions": {  
  "additionalDirectories": [  
    "../shared-lib",  
    "~/reference/design-docs"  
  ] } }
```

효과

해당 경로가 작업 디렉토리처럼 접근 가능

acceptEdits의 자동 수락 범위에도 포함

CLI: `claude --add-dir ../shared-lib`

추가 경로의 `.claude/agents/`도 로드 (Ch.2)

범위 확장

규칙 아닌 키

`--add-dir`

CLI 동등물

`acceptEdits`

자동 수락 연동

구성 로드

agents 동반 스캔

조직 정책과의 상호작용

Chapter 3 강제 계층 위에서

UNDER MANAGED

내 allow는 조직 deny를 이길 수 없습니다.

병합은 합집합이지만 deny 우선 원칙이 관리 차단을 절대선으로 만듭니다.

개발자가 할 수 있는 것

자기 allow로 마찰 줄이기

프로젝트 표준 allow, ask 제안

additionalDirectories 확장

모드 선택 (차단 안 된 범위)

조직이 강제하는 것

managed deny: 제거 불가

allowManagedPermissionRulesOnly

disableBypassPermissionsMode

disableAutoMode 등 모드 봉쇄

/permissions

규칙의 대화형 창구

~/proj \$ claude

> /permissions

현재 유효 규칙을 스코프별로 열람

allow, ask, deny 추가와 삭제

어느 파일에서 온 규칙인지 출처 표시

세션 중 빠른 허용의 정석

확인 프롬프트에서 항상 허용 선택

-> settings.local.json에 자동 기록

팀 표준 승격: local에서 검증 후 project로 이동

출처 표시

스코프 진단

항상 허용

local 자동 기록

승격 흐름

local -> project

즉시 반영

위치 연동

설계 예시 / 팀 표준

웹 서비스 저장소의 permissions

.claude/settings.json (permissions)

```
{ "permissions": {
  "allow": [
    "Bash(npm run lint)", "Bash(npm run test *)",
    "Bash(npm run build)", "Bash(git diff *)",
    "Bash(git log *)", "Bash(git status)"
  ],
  "ask": ["Bash(git push *)", "Bash(npm publish *)"],
  "deny": [
    "Read(./env*)", "Read(./secrets/**)",
    "Bash(curl *)", "Bash(rm -rf *)"
  ]
} }
```

빌드 3종

무확인 생산성

조회 git

전면 allow

배포류 ask

확인 유지

비밀 deny

3종 세트

흔한 실수

현장에서 걸리는 지점

실수

- Bash(npm run test)가 인자 불자 불일치
- deny에 .env만 넣고 .env.local 누락
- 허용 과다로 ask의 의미 소실
- local에 쌓인 규칙을 팀 표준에 미반영
- 권한만으로 네트워크 봉쇄 기대

교정

- 인자 허용은 명령 뒤 공백 별표
- .env* 와 secrets/** 패턴화
- 회색 지대만 ask, 명확 위험은 deny
- 주기적으로 local 검토 후 승격
- 봉쇄는 sandbox 도메인과 병행 (Ch.3)

Part 2 Recap

Permissions 핵심 정리

KEY TAKEAWAY

한 형식 Tool(specifier), 세 동사, deny 우선.

모드는 무일치의 기본 거동이고, auto 분류기는 산문 규칙으로 조율합니다.

핵심 정리

- deny > allow, ask > 모드 폴백
- Bash 그룹과 gitignore식 경로
- 모드 6종, dontAsk는 fail-safe
- autoMode 산문 규칙 + \$defaults
- local 검증 후 project 승격

다음 파트

- Part 3 Hooks 아키텍처
- 30 이벤트 카탈로그
- 5 핸들러 타입
- matcher와 if 문법
- 입출력과 결정 제어

CHAPTER 04 / PART 03

Hooks 아키텍처

30개 이벤트, 5가지 핸들러

- 수명주기 3 케이던스와 이벤트 카탈로그
- 구성 3단 중첩과 matcher 문법
- if 필드와 입력 JSON
- 결정 출력과 exec form

Hook이란 / 5가지 핸들러

수명주기 지점에 쏘는 자동 실행

HOOKS 2026

혹은 세션 수명주기의 특정 지점에서 자동 실행되는 핸들러입니다.

셸 명령을 넘어 HTTP, MCP 도구, LLM 프롬프트, 에이전트까지 다섯 타입으로 확장되었습니다.

command

셸 명령

stdin JSON 수신, exit와 stdout
으로 응답.
고전이자 중심.

http

HTTP POST

이벤트 JSON을 URL로 전송,
응답 본문이 결정.

mcp_tool

MCP 도구

연결된 서버의 도구를 핸들러로
직접 호출.

prompt / agent

LLM 판정

모델 단발 판정 또는
도구 가진 서브에이전트 검증.

5 types

핸들러 타입

30 events

이벤트 수

병렬 실행

매칭 혹 동시

중복 제거

동일 핸들러 자동

수명주기 3 케이던스

세션당, 턴당, 도구 호출당

THREE CADENCES

이벤트는 발화 빈도로 세 층입니다.

세션당 한 번, 사용자 턴마다, 그리고 에이전틱 루프의 도구 호출마다입니다.

SESSION

세션당

SessionStart /
SessionEnd, Setup

TURN

턴당

UserPromptSubmit,
Stop, StopFailure

TOOL

도구 호출당

PreToolUse,
PostToolUse 계열

ASYNC

상시 감시

FileChanged,
ConfigChange,
CwdChanged

이벤트 카탈로그 1 / 코어

가장 자주 쓰는 여덟

PreToolUse	도구 실행 전, 차단 가능	가드의 분진
PostToolUse	도구 성공 후	포맷터, 후처리
PostToolUseFailure	도구 실패 후	실패 대응 분기
UserPromptSubmit	프롬프트 제출 시	컨텍스트 주입
Stop / StopFailure	턴 종료 / API 오류 종료	알림, 오류 매치
SessionStart / End	세션 시작(재개, clear 매치) / 종료	환경 준비, 정리

이벤트 카탈로그 2 / 에이전트와 권한

루프 내부의 확장 이벤트

SubagentStart / Stop

서브에이전트 생성, 종료 (타입 매치)

Chapter 2 관측

PermissionRequest

권한 대화상자 표시 시점

승인 흐름 관측

PermissionDenied

auto 분류기 거부 시, retry 지시 가능

재시도 정책

PostToolBatch

병렬 도구 묶음 완료 후

배치 단위 후처리

TaskCreated / Completed

작업 항목 생성, 완료

진행 추적 연동

TeammateIdle

팀 동료 유희 직전

Agent Teams 연계

이벤트 카탈로그 3 / 환경과 컨텍스트

세션 밖 변화에 반응

FileChanged	감시 파일 변경 (매처가 파일명)	.env 변경 감지
ConfigChange	설정 파일 변경 (소스 매처)	정책 변경 알림
CwdChanged	작업 디렉토리 이동	direnv 류 연동
InstructionsLoaded	CLAUDE.md, rules 로드 시	지침 적용 추적
Pre / PostCompact	압축 전후 (manual, auto 매처)	상태 보존, 재주입
Worktree Create / Remove	워크트리 생성, 제거 커스텀	기본 git 동작 대체

구성 3단 중첩

이벤트, 매처 그룹, 핸들러

```
.claude/settings.json (hooks)

{ "hooks": {
  "PreToolUse": [                // 1 이벤트
    { "matcher": "Bash",          // 2 매처 그룹
      "hooks": [                 // 3 핸들러 목록
        { "type": "command",
          "if": "Bash(rm *)",
          "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/block-rm.sh",
          "args": [] }
      ] } ] } }

# 위치: user, project, local settings + managed,
#       plugin hooks.json, skill/agent frontmatter
```

3단 중첩

event, group, handler

6 위치

정의 가능 지점

PROJECT_DIR

경로 플레이스홀더

즉시 반영

위치 리로드

matcher 문법

문자 구성이 해석 방식을 결정

"*", "", 생략	전체 매칭	이벤트 전 발화
영숫자 _ - 공백 ,	정확 문자열 (리스트 구분자 또는 ,)	Edit Write, Bash
그 외 문자 포함	JS 정규식, 비앵커	^Notebook, mcp__s__.*
mcp 주의	mcp__memory는 정확 문자열이라 무매칭	mcp__memory__.* 필수
앵커 권장	Edit.*는 NotebookEdit도 매칭	^Edit\$로 전체 일치
이벤트별 대상	도구명, 에이전트 타입, 트리거, 파일명 등	카탈로그 표 참조

if 필드

권한 규칙 문법으로 2차 필터

if narrowing

```
{ "matcher": "Bash",  
  "hooks": [{ "type": "command",  
    "if": "Bash(git *)",    // 권한 규칙 문법 1개  
    "command": "..."}] }
```

Bash 서브커맨드까지 검사

npm test && git push -> git * 매칭, 실행

echo \$(rm -rf /) -> rm * 매칭, 실행

VAR=x git push -> 선행 대입 제거 후 매칭

파싱 불가면 fail-open: 강제는 권한, 혹은 보조

규칙 1개

조합 문법 없음

서브커맨드

&&, \$() 내부 검사

스폰 절약

불일치 시 미실행

fail-open

강제 수단 아님

입력 JSON

핸들러가 받는 공통 필드

stdin / POST body

```
{
  "session_id": "...",
  "prompt_id": "...",          // OTel 이벤트와 상관 연결
  "transcript_path": "~/.claude/projects/.../*.jsonl",
  "cwd": "/home/dev/proj",
  "permission_mode": "default",
  "effort": { "level": "high" },
  "tool_name": "Bash",         // 도구 이벤트 한정
  "tool_input": { "command": "rm -rf /tmp/build" }
}
# command 혹은 stdin, http 혹은 POST 본문으로 동일 JSON
```

prompt_id

텔레메트리 상관키

mode/effort

런타임 상태 동봉

tool_input

이벤트별 상세

jq 파싱

핸들러 관용구

결정 출력

침묵은 승인이 아니다

● ● ● decision output

```
# 방식 1: JSON 결정 (권장)
jq -n '{ hookSpecificOutput: {
  hookEventName: "PreToolUse",
  permissionDecision: "deny",
  permissionDecisionReason: "Destructive command" } }'
```

방식 2: exit code

- # exit 0 + 무출력 = 결정 없음 (정상 권한 흐름 계속)
- # exit 2 + stderr = 차단, 사유가 Claude에게 전달

사용자 표시: systemMessage / 알림: terminalSequence

혹은 무단말 세션: /dev/tty 접근 불가 (v2.1.139+)

deny JSON

정밀 결정 형식

exit 2

고전 차단 유효

침묵=무결정

승인 아님

no tty

메시지는 JSON으로

exec form vs shell form

args 유무가 실행 방식을 가른다

TWO FORMS

args가 있으면 셸 없이 실행 파일을 직접 스폰하고, 없으면 셸이 문자열을 해석합니다.

경로 플레이스홀더에는 exec form이 권장됩니다.

exec form (args 있음)

- command는 실행 파일, args가 인자 벡터
- 셸 미개입: 인용, 공백 걱정 없음
- 플레이스홀더가 문자열 그대로 치환
- Windows는 .cmd 심 불가, node + 스크립트로

shell form (args 없음)

- 문자열을 sh -c 등 셸이 해석
- 파이프, &&, 리다이렉트 사용 가능
- 플레이스홀더는 따옴표로 감싸기
- shell: powershell로 셸 선택

async와 timeout

블로킹을 다스리는 필드들

timeout 기본	command, http, mcp_tool 600초	prompt 30, agent 60
이벤트별 하향	UserPromptSubmit 30초, MessageDisplay 10초	체감 지연 보호
async: true	백그라운드 실행, 턴을 막지 않음	느린 후처리
asyncRewake	백그라운드 + exit 2 시 Claude 깨움	장기 감시 실패 보고
statusMessage	실행 중 스피너 문구 커스텀	사용자 체감 개선
once	세션당 1회 후 제거	스킬 프론트매터 전용

Part 3 Recap

Hooks 아키텍처 핵심 정리

KEY TAKEAWAY

30 이벤트를 3 케이던스로 이해하고, 매처와 if로 거르며, JSON 결정으로 답합니다.

침묵은 승인이 아니고, 강제는 권한의 묵입니다.

핵심 정리

5 핸들러: command, http, mcp_tool, prompt, agent

매처: 문자 구성이 해석 결정

if: 서브커맨드 검사, fail-open

결정: permissionDecision JSON

exec form + args 권장

다음 파트

Part 4 Hooks 실전

HTTP, MCP tool, prompt 혹

포맷터, 가드, 알림 레시피

FileChanged 반응형 환경

/hooks 메뉴와 디버깅

CHAPTER 04 / PART 04

Hooks 실전

새 핸들러와 검증된 레시피

- HTTP, MCP tool, prompt 핸들러 실전
- 포맷터, 가드, 알림 3대 레시피
- FileChanged 반응형 환경
- /hooks 메뉴와 디버깅

HTTP 훅

중앙 서비스로 이벤트를 쏜다

http handler

```
{ "hooks": { "PreToolUse": [  
  { "matcher": "Bash",  
    "hooks": [{ "type": "http",  
                 "url": "https://hooks.corp.example/pre-tool",  
                 "timeout": 30,  
                 "headers": { "Authorization": "Bearer $HOOK_TOKEN" },  
                 "allowedEnvVars": ["HOOK_TOKEN"] }] } ] } }
```

이벤트 JSON이 POST 본문으로 전송

차단: 2xx + permissionDecision deny 본문

비2xx, 타임아웃은 비차단 오류로 계속 진행

POST JSON

전송 형식

allowedEnvVars

헤더 보관 허용

2xx+deny

차단 유일 경로

allowedHttpHookUrls

조직 목적지 통제

mcp_tool 후

연결된 서버의 도구를 핸들러로

mcp_tool handler

```
{ "hooks": { "PostToolUse": [
  { "matcher": "Write|Edit",
    "hooks": [{ "type": "mcp_tool",
      "server": "security",
      "tool": "security_scan",
      "input": { "file_path": "${tool_input.file_path}" } } ] }
] } }
```

입력에 \${경로} 치환으로 이벤트 필드 전달
서버는 이미 연결 상태여야 함 (OAuth 유발 없음)
SessionStart, Setup에서는 연결 전이라 첫 실행 유의

server + tool

지정 2필드

`${path}` 치환

이벤트 필드 주입

연결 전제

인증 유발 없음

isError 비차단

오류 의미론

prompt, agent 후

판정을 모델에게 맡긴다

prompt / agent handlers

```
{ "type": "prompt",  
  "prompt": "이 도구 호출이 마이그레이션 파일을 건드리면  
  deny하라. 입력: $ARGUMENTS",  
  "model": "haiku" }
```

```
{ "type": "agent",  
  "prompt": "diff를 읽고 스키마 파괴 변경 여부를 검증 후  
  결정하라. 입력: $ARGUMENTS" }
```

prompt: 단발 판정 (기본 fast model, timeout 30)

agent: Read, Grep 등 도구로 확인 후 판정 (실험적)

\$ARGUMENTS

입력 JSON 자리

정규식 너머

의미 기반 판정

haiku 권장

지연, 비용 통제

agent 실험적

변경 가능성

Recipe 1 / Auto Format

편집 직후 자동 포맷

.claude/settings.json + hook

```
{ "hooks": { "PostToolUse": [
  { "matcher": "Edit|Write",
    "hooks": [{ "type": "command",
      "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/format.sh",
      "args": [], "timeout": 30 } ] } ] }
```

```
#!/bin/bash # .claude/hooks/format.sh
FILE=$(jq -r '.tool_input.file_path')
case "$FILE" in
  *.ts|*.tsx) npx prettier --write "$FILE" ;;
  *.py)      ruff format "$FILE" ;;
esac
```

Edit|Write

발화 지점

exec form

args 배열 사용

확장자 분기

다언어 대응

커밋 공유

팀 표준 확

Recipe 2 / 보호 가드

공식 예시 block-rm의 완성형

.claude/hooks/block-rm.sh

```
#!/bin/bash
COMMAND=$(jq -r '.tool_input.command')
if echo "$COMMAND" | grep -q 'rm -rf'; then
  jq -n '{ hookSpecificOutput: {
    hookEventName: "PreToolUse",
    permissionDecision: "deny",
    permissionDecisionReason:
      "Destructive command blocked by hook" } }'
else
  exit 0  # 무결정: 정상 권한 흐름 계속
fi
# 등록: matcher Bash + if "Bash(rm *)" (스폰 절약)
```

deny JSON

사유 동봉 차단

if 필터

rm 계열만 스폰

무결정 exit 0

그 외 통과

권한 병행

deny 규칙과 이중

Recipe 3 / 알림

기다림을 끝내는 후

```
notification hooks

{ "hooks": {
  "Notification": [
    { "matcher": "permission_prompt|agent_needs_input",
      "hooks": [{ "type": "command",
        "command": "notify-send 'Claude가 입력을 기다립니다'" }] } ],
  "Stop": [
    { "hooks": [{ "type": "http",
      "url": "https://hooks.slack.com/services/T/B/x",
      "timeout": 10 }] } ]
  } }
# Notification 매처: permission_prompt, idle_prompt,
# agent_completed 등 유형별 선별
```

유형 매치

알림 선별

Stop + http

턴 종료 슬랙

terminalSequence

벨, 제목 대안

장기 작업

체감 효용 최대

Recipe 4 / 반응형 환경

FileChanged와 CwdChanged

reactive environment

```
{ "hooks": {  
  "FileChanged": [  
    { "matcher": ".envrc|.env",  
      "hooks": [{ "type": "command", "async": true,  
                  "command": "direnv allow && echo env reloaded" }] } ],  
  "CwdChanged": [  
    { "hooks": [{ "type": "command",  
                  "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/on-cd.sh" }] } ]  
  } }
```

FileChanged 매처 = 감시할 파일명 목록
cd 명령마다 환경 재정렬, direnv 류와 공합

파일명 매처

감시 목록 선언

async

턴 비차단

cd 반응

모노레포 환경 전환

ConfigChange

정책 감시 자매 혹

Recipe 5 / Setup과 컨텍스트 주입

CI 준비와 턴 시작 정보

● ● ● setup + prompt context

```
# CI 1회 준비: claude --init-only 가 Setup 발화
{ "hooks": { "Setup": [
  { "matcher": "init",
    "hooks": [{ "type": "command",
      "command": "npm ci && cp .env.ci .env" }] ] } }

# 프롬프트마다 최신 상태 주입
{ "hooks": { "UserPromptSubmit": [
  { "hooks": [{ "type": "command",
    "command": "git status --short | head -5" }] ] } } }

# stdout이 컨텍스트로 추가됨 (timeout 30초 주의)
```

--init-only

Setup 트리거

stdout 주입

UserPromptSubmit

30초 상한

제출 경로 보호

SessionStart

세션 단위 대안

/hooks 메뉴와 디버깅

무엇이 어디서 왔는지

OBSERVABILITY

/hooks는 구성된 전 hook의 읽기 전용 브라우저입니다.

타입 접두와 소스 라벨로 출처를 추적하고, 문제 시 disableAllHooks로 이분 탐색합니다.

BROWSE

hook 브라우저

이벤트별 개수, 매치, 핸들러
상세 열람.

SOURCE

출처 라벨

User, Project, Local,
Plugin, Session, Built-in.

EDIT

수정 경로

메뉴는 읽기 전용, 편집은 파일
또는 Claude에게.

DEBUG

이분 탐색

disableAllHooks로 hook 원인
여부부터 판정.

[type] 접두

5 타입 표기

6 소스

출처 추적

read-only

메뉴 성격

doctor 병행

Part 8 진단

Part 4 Recap

Hooks 실전 핵심 정리

KEY TAKEAWAY

포맷터, 가드, 알림이 3대 기본기입니다.

HTTP는 중앙화, mcp_tool은 무스크립트 연동, prompt는 의미 판정이라는 새 카드를 상황에 맞게 뽑으세요.

핵심 정리

- HTTP: 2xx+deny만 차단
- mcp_tool: \${} 치환, 연결 전제
- prompt: 의미 판정, haiku
- 레시피 5: 포맷, 가드, 알림, 반응형, 주입
- /hooks 출처 추적 + 전체 비활성

다음 파트

- Part 5 MCP 구성
- 3 프리미티브와 4 전송
- 설치와 스코프
- OAuth와 도구 검색
- 컨텍스트 예산

CHAPTER 04 / PART 05

MCP 구성

외부 세계로 열린 표준 다리

- 3 프리미티브와 4 전송
- 설치 CLI와 스코프 3종
- .mcp.json과 OAuth
- 도구 검색과 컨텍스트 예산

MCP와 3 프리미티브

도구, 리소스, 프롬프트

MODEL CONTEXT PROTOCOL

MCP는 모델과 외부 시스템을 잇는 개방 표준입니다.

서버는 세 종류의 능력을 노출하고, Claude Code가 클라이언트로서 이를 소비합니다.

TOOLS

도구

모델이 호출하는 동작,
mcp__서버__도구 이름으로
등장.

RESOURCES

리소스

읽을 수 있는 데이터, @ 멘션
으로 컨텍스트에 첨부.

PROMPTS

프롬프트

서버 제공 정형 명령, 슬래시로
노출.

ROLE

역할 구도

Claude Code = 클라이언트,
연결 대상 = 서버.

개방 표준
생태계 호환

mcp__
도구 이름 접두

@ 멘션
리소스 소비

Ch.6 구축
서버 제작은 SDK

전송 4종

서버와 이어지는 네 길

stdio

로컬 프로세스를 스폰해 표준입출력 통신

npx 배포 서버의 표준

http

원격 HTTP 엔드포인트, 요청 응답

관리형 서비스의 표준

sse

서버 전송 이벤트 스트림

레거시 원격, http로 이행 추세

ws

웹소켓 양방향

실시간성 요구 서버

선택 기준

로컬 도구는 stdio, SaaS는 http

URL이 있으면 원격 계열

설치 / claude mcp add

전송별 추가 문법

install commands

```
# stdio: -- 뒤가 실행 명령
$ claude mcp add playwright -- npx -y @playwright/mcp@latest

# 원격: transport 지정
$ claude mcp add --transport http github \
  https://api.githubcopilot.com/mcp/

# 스코프 지정 (-s): local(기본) | project | user
$ claude mcp add -s project sentry -- npx -y @sentry/mcp

$ claude mcp list / get <name> / remove <name>
```

-- 구분자

stdio 명령 경계

--transport

원격 지정

-s 스코프

local 기본

list/get/remove

관리 3종

스코프 3종과 저장 위치

Part 1 파일 지도의 MCP 편

local (기본)	~/.claude.json의 프로젝트별 항목	이 저장소의 나만, 비공유
project	.mcp.json (저장소 루트)	커밋 공유, 팀 표준
user	~/.claude.json 전역 항목	내 전 프로젝트
우선순위	local > project > user (동명 시)	설정 계층과 동형
신뢰 확인	project 서버는 최초 사용 승인 절차	저장소별 구성 보호

.mcp.json

팀 공유 서버 정의와 변수 확장

.mcp.json (project scope)

```
{
  "mcpServers": {
    "corp-wiki": {
      "type": "http",
      "url": "https://wiki-mcp.corp.example/mcp",
      "headers": { "Authorization": "Bearer ${WIKI_TOKEN}" }
    },
    "db": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@corp/db-mcp"],
      "env": { "DB_HOST": "${DB_HOST:-localhost}" } } } }
```

`${VAR}`

환경변수 확장

`:-기본값`

미설정 폴백

비밀 외부화

파일에 토큰 금지

커밋 대상

팀 표준 파일

OAuth 인증 흐름

원격 서버 로그인의 표준

STEP 1

추가

http 서버 add, 미인증 상태

STEP 2

/mcp

서버 선택,
Authenticate 실행

STEP 3

브라우저

제공자 로그인, 권한 동의

STEP 4

토큰 저장

자동 갱신, 이후 무개입

재인증과 로그아웃도 /mcp에서, 토큰은 로컬 안전 저장

/mcp

인증 창구

브라우저 위임

자격 미노출

자동 갱신

토큰 수명 관리

clear auth

재인증 경로

사용하기

도구, 리소스, 프롬프트의 소비 문법

● ● ● ~/proj \$ claude

```
# 도구: 자연어로, 권한 규칙은 mcp__ 이름
> github에서 이 저장소 열린 이슈 요약해줘
# 내부적으로 mcp__github__list_issues 호출

# 리소스: @ 멘션으로 컨텍스트 첨부
> @corp-wiki:onboarding/backend 읽고 셋업 도와줘

# 서버 프롬프트: 슬래시로 호출
> /mcp__github__pr_review 123

# 권한: allow에 mcp__github 또는 개별 도구
```

자연어

도구 호출 경로

@서버:경로

리소스 멘션

/mcp__

서버 프롬프트

Part 2 연동

권한 규칙 대상

/mcp 관리 UI

연결의 상황판

MCP DASHBOARD

/mcp는 서버 목록과 연결 상태, 도구 수, 인증을 한 화면에서 다룹니다. 서버가 안 보이면 여기부터 봅니다.

STATUS

연결 상태

connected, failed 등 서버 별 헬스 표시.

DETAIL

상세 열람

도구, 리소스 목록과 스코프 출처 확인.

AUTH

인증 관리

OAuth 로그인, 재인증, 로그아웃.

TOGGLE

활성 토글

세션에서 서버 켜고 끄기.

헬스

연결 진단 1선

출처

스코프 추적

인증

OAuth 창구

Part 8

심화 진단 연결

claude.ai 커넥터

웹에서 연결한 서버가 CLI로

CLOUD CONNECTORS

claude.ai에서 연결한 커넥터가 Claude Code 세션에도 자동 등장합니다.

조직과 저장소는 설정 키로 이를 통제할 수 있습니다.

SYNC

자동 동기

웹 커넥터가 CLI 세션에 자동 폐치, 연결.

OPT-OUT

저장소 거부

disableClaudeAiConnectors를 프로젝트에 커밋 가능.

PRECEDENCE

true 우선

어느 소스든 true면 차단, false로 재개방 불가.

MANAGED

조직 결합

managed-mcp.json 배포 시 기본 배제, allowAll 키로 병행.

자동 등장

기본 동작

true 승리

차단 우선 규칙

deniedMcpServers

개별 거부 대안

Ch.3 연결

managed 통제

도구 검색 / Tool Search

대량 도구 시대의 해법

ON-DEMAND TOOLS

서버가 많아지면 도구 정의만으로 컨텍스트가 잠식됩니다.

도구 검색은 설명 대신 검색 인덱스를 두고, 필요한 도구만 온디맨드로 로드합니다.

PROBLEM

정의 비대

수백 도구의 스키마가 시작
컨텍스트를 점령.

HOW

지연 로드

도구는 인덱스로만 존재, 검색 후 정의 로드.

WHEN

발동 조건

도구 수가 임계 초과 시 자동 전환.

EFFECT

체감

다서버 환경에서 시작 토큰
급감, 정확한 도구 선택.

인덱스

정의 대체물

온디맨드

로드 시점

자동 전환

임계 기반

SDK 문서

tool-search 상세

컨텍스트 예산

서버는 공짜가 아니다

CONTEXT BUDGET

연결된 서버의 도구 설명은 컨텍스트를 소비합니다.

도구 검색이 완화하지만, 스코프 설계로 노출 자체를 줄이는 것이 근본입니다.

잠식 신호

/context에서 MCP tools 비중 확인

시작부터 컨텍스트 수십 퍼센트 점유

안 쓰는 서버가 user 스코프에 상주

무거운 서버를 전 프로젝트에 노출

다이어트 처방

프로젝트별 필요 서버만 .mcp.json

개인 상주는 최소, 나머지는 온디맨드 add

무거운 서버는 서브에이전트 mcpServers로 (Ch.2)

/mcp 토크로 세션 단위 오프

Part 5 Recap

MCP 구성 핵심 정리

KEY TAKEAWAY

3 프리미티브, 4 전송, 3 스코프가 문법의 전부입니다.

팀 표준은 .mcp.json 커밋, 인증은 OAuth 위임, 규모는 도구 검색과 스코프 설계로 다스립니다.

핵심 정리

- 도구, 리소스, 프롬프트 소비 문법
- stdio는 --, 원격은 --transport
- local/project/user + 신뢰 승인
- \${VAR} 확장, 비밀 외부화
- 도구 검색 + 컨텍스트 예산

다음 파트

- Part 6 MCP 운영과 보안
- 서버 카탈로그
- 신뢰 모델과 인젝션
- 조직 통제 요약
- 트러블슈팅

CHAPTER 04 / PART 06

MCP 운영과 보안

연결의 규율

- 인기 서버 카탈로그
- 신뢰 모델과 프롬프트 인젝션
- 조직 통제와 흑 결합
- 성능과 트러블슈팅

인기 서버 카탈로그

현장에서 검증된 출발선

github	이슈, PR, 코드 검색과 조작	http, OAuth
playwright	브라우저 구동, E2E와 스크린샷	stdio, npx
sentry	오류 이벤트 조회와 분석	http, OAuth
slack	채널 검색, 메시지 발신	http, OAuth
postgres 계열	스키마 조회, 쿼리 실행	stdio, 자격은 env
notion, linear, figma	문서, 이슈, 디자인 컨텍스트	http, OAuth

사내 MCP 서버

만드는 쪽의 개요 (상세는 Ch.6)

BUILD YOUR OWN

사내 위키, 배포, 티켓 시스템을 서버로 노출하면 조직 컨텍스트가 도구가 됩니다.

제작은 Agent SDK 계열 문서와 챕터 6에서 심화합니다.

WHAT

노출 대상

위키 검색, 배포 상태, 티켓
CRUD, 사내 API.

STACK

제작 스택

TypeScript, Python MCP
SDK로 도구 정의.

SHIP

배포 형태

studio 패키지 또는 사내
http 엔드포인트.

GOV

운영 결합

OAuth 인증, allowlist 등
재, 버전 관리.

3 프리미티브

설계 단위

http 권장

다수 사용자 배포

Ch.6

제작 심화 위치

Part 4 연계

네트워크 준비 (Ch.3)

신뢰 모델

서드파티 서버와 프롬프트 인젝션

TRUST BOUNDARY

서버가 돌려주는 텍스트는 모델이 읽는 입력입니다.

악성 서버나 오염된 데이터가 지시문을 심을 수 있으므로, 서버 선택 자체가 보안 결정입니다.

위험 지점

- 미검증 서버의 도구 설명, 응답에 지시 삽입
- 외부 데이터(이슈, 문서) 경유 간접 인젝션
- 과도한 쓰기 권한을 가진 도구
- 토큰이 서버 측에 과하게 위임됨

방어 원칙

- 공식, 사내 제작 서버 우선 채택
- 프로젝트 서버 최초 사용 승인 신중히
- 쓰기 도구는 ask, 민감 도구는 deny
- 외부 데이터 처리 시 혹 검증 결합

조직 통제 요약

Chapter 3 managed 키의 MCP 편

<code>allowedMcpServers</code>	승인 서버 화이트리스트	빈 배열 = 전면 봉쇄
<code>deniedMcpServers</code>	명시 차단, allowlist보다 우선	사고 대응 즉시 배포
<code>allowManagedMcpServersOnly</code>	managed 목록만 유효	최고 수준 통제
<code>managed-mcp.json</code>	조직 표준 서버 정의 배포	커넥터 기본 배제
<code>disableSideloadFlags</code>	--mcp-config 등 우회 거부	v2.1.193+ (Ch.3)
적용 범위	서브에이전트 인라인 정의 포함	Ch.2 우회 봉쇄

훅 결합

mcp__ 매처로 감사와 검증

hooks x mcp

```
{ "hooks": { "PreToolUse": [  
  { "matcher": "mcp__.*__(write|create|delete).*",  
    "hooks": [{ "type": "command",  
                  "command": "${CLAUDE_PROJECT_DIR}/.claude/hooks/mcp-write-guard.sh",  
                  "args": [] } ] },  
  { "matcher": "mcp__corp-wiki__.*",  
    "hooks": [{ "type": "command", "async": true,  
                  "command": "jq -c '{ts:now,tool:.tool_name}' >> ~/mcp-audit.jsonl" } ] }  
] } }
```

쓰기 계열 정규식 가드 + 서버 전체 비동기 감사

쓰기 정규식

위험 동사 매칭

서버 감사

mcp_s__.* 전량

async 기록

지연 없는 감사

Part 3 재사용

매처 문법 그대로

성능 관점

연결이 늘 때 생기는 비용

STARTUP

기동 지연

stdio 다중 스폰과 원격 핸드셰이크가 시작을 늦춤.

CONTEXT

컨텍스트

도구 정의 비대, 도구 검색과 스코프로 완화 (Part 5).

STRICT

구성 고정

--strict-mcp-config로 지정 구성 외 로드 배제.

BARE

무장식 기동

--bare로 MCP 등 확장 없이 최소 기동.

스폰 비용

stdio 다중화

인덱스화

도구 검색 효과

--strict-mcp-config

CI 재현성

--bare

진단, 벤치 용도

MCP 트러블슈팅

증상에서 원인으로

서버 안 보임

스코프, 신뢰 승인 미완

/mcp 목록과 출처 확인

connection failed

명령 경로, URL, 네트워크

단독 실행, curl 검증

401, 인증 오류

OAuth 만료, 토큰 변수 미설정

/mcp 재인증, env 확인

도구 호출 거부

권한 규칙 미허용

/permissions에 mcp__ 추가

조직에서 차단

allowlist 미등재

관리자 등재 요청 (Ch.3)

느린 시작

서버 과다, 무거운 stdio

스코프 축소, 도구 검색

운영 모범 사례

연결의 규율 네 가지

RULE 1

표준은 커밋

팀 서버는 .mcp.json으로,
개인 상주는 최소로.

RULE 2

비밀 외부화

토큰은 \${VAR}와 볼트,
파일에는 절대 금지.

RULE 3

쓰기는 관문

쓰기 도구 ask + 혹 가드,
읽기는 넉넉히.

RULE 4

분기 정리

미사용 서버 제거, 버전과 인증
상태 점검.

커밋 표준

팀 정합

0 secrets

파일 원칙

read easy

write hard

분기 점검

위생 주기

Part 6 Recap

MCP 운영과 보안 핵심 정리

KEY TAKEAWAY

서버 응답은 모델의 입력입니다.

서버 선택, 조직 allowlist, 혹은 가드의 3중 방어에 성능 다이어트를 더하면 연결의 규율이 완성됩니다.

핵심 정리

- 카탈로그: SaaS는 OAuth http
- 인젝션: 응답도 입력이다
- 3중 방어: 권한, 조직, 혹은
- `--strict-mcp-config`, `--bare`
- 표준 커밋 + 비밀 외부화

다음 파트

- Part 7 Commands와 Skills
- 확장 수단 선택 지형
- 커스텀 명령 문법
- SKILL.md 구조
- 팀 라이브러리

CHAPTER 04 / PART 07

Commands와 Skills

반복을 명령으로 굳히기

- 확장 수단 선택 지형
- 커스텀 명령 문법과 frontmatter
- SKILL.md 구조와 고급 옵션
- 팀 라이브러리와 선택 가이드

확장 지형 / 무엇으로 확장하나

여섯 수단의 한 줄 판별 (features-overview)

CLAUDE.md

항상 알아야 할 프로젝트 지식

상시 컨텍스트 (Ch.1)

Command / Skill

반복 작업의 정형 절차

사용자가 호출, 본 파트

Subagent

격리 컨텍스트의 전담 워커

위임 실행 (Ch.2)

Hook

수명주기 자동 반응

이벤트 구동 (Part 3, 4)

MCP

외부 시스템 연결

도구 확장 (Part 5, 6)

Plugin

위 전부의 배포 묶음

마켓 유통

커스텀 명령 기본

Markdown 한 장이 슬래시 명령

.claude/commands/summarize.md

```
---  
description: 최근 변경을 한국어로 요약  
---  
git log --oneline -10 결과를 바탕으로 이번 주 변경 사항을 다섯 줄로 요약하라.  
  
# 사용  
$ claude  
> /summarize  
  
# 위치: .claude/commands/(프로젝트), ~/.claude/commands/(개인)  
# 파일명이 곧 명령 이름, 하위 폴더는 네임스페이스
```

파일명 = 명령

생성 규칙

2 위치

프로젝트, 개인

폴더 네임스페이스

/dir:name

즉시 반영

위치 리로드

인자 받기

\$ARGUMENTS와 위치 인자

```
.claude/commands/fix-issue.md

---
description: 이슈 번호로 수정 착수
argument-hint: "<이슈번호> [우선순위]"
---

이슈 #$1 을 조사해 수정하라.
우선순위는 $2, 전체 지시: $ARGUMENTS

# 사용
> /fix-issue 123 high
# $1=123, $2=high, $ARGUMENTS="123 high"
# argument-hint가 자동완성 힌트로 표시
```

\$ARGUMENTS

전체 문자열

\$1, \$2

위치 인자

argument-hint

자동완성 안내

미전달 = 공백

옵션 인자 처리

frontmatter 옵션

명령의 메타데이터

description	명령 목록과 모델 판단에 쓰이는 설명	필수 습관
argument-hint	자동완성에 표시할 인자 안내	<필수> [선택] 관례
allowed-tools	이 명령 실행 중 허용 도구 제한	최소 권한 원칙
model	이 명령 전용 모델 지정	무거운 분석은 opus
hooks	명령 활성화 중 스코프 혹은 (once 지원)	Part 3 프론트매터 혹은
disable-model-invocation	모델의 자동 호출 금지	사용자 전용 명령

컨텍스트 수집

!실행과 @참조

.claude/commands/pr-context.md

```
---
description: PR 컨텍스트를 모아 리뷰 준비
allowed-tools: Bash(git *), Read
---
```

현재 상태

- 브랜치: !`git branch --show-current`
- 변경: !`git diff --stat main...HEAD`

기준 문서

@docs/review-checklist.md

위를 바탕으로 리뷰 관점 5가지를 제시하라.

!`cmd`

인라인 실행 삽입

@경로

파일 첨부

실행 시점 평가

항상 최신 상태

disableSkillShellExecuti

on
조식 차단 키 존재

Skills 구조

명령의 성장형, 폴더와 리소스

.claude/skills/deploy-check/

deploy-check/

SKILL.md # 본문 + frontmatter

checklist.md # 동봉 리소스

scripts/verify.sh # 동봉 스크립트

--- SKILL.md ---

name: deploy-check

description: 배포 전 검증 절차. 배포 언급 시 사용.

checklist.md의 항목을 순서대로 점검하고 scripts/verify.sh 실행 결과를 해석하라.

폴더 단위

리소스 동봉

SKILL.md

진입 본문

모델 자동 발동

description 기반

/이름 호출

명령 검용

스킬 고급 옵션

실행 방식과 수명 제어

context: fork	격리 컨텍스트에서 실행 후 요약 귀환	무거운 절차의 본진 보호
allowed-tools	스킬 활성화 중 도구 제한	절차 단위 최소 권한
hooks + once	활성 중 스코프 혹은, 세션 1회 옵션	준비 작업에 최적
disable-model-invocation	자동 발동 금지, /호출 전용	위험 절차 안전판
번들 스킬	기본 동봉 절차들, 조직 비활성 가능	disableBundledSkills
플러그인 배포	스킬을 플러그인으로 묶어 유통	마켓, 팀 배포

실전 명령 1 / PR 리뷰

수집, 위임, 판정의 한 줄화

● ● ● .claude/commands/pr-review.md

description: 현재 브랜치 PR 리뷰 패키지 생성

argument-hint: "[base브랜치]"

allowed-tools: Bash(git *), Read, Grep, Agent

기준: !`git merge-base HEAD \${1:-main} 2>/dev/null || echo main`

변경: !`git diff --stat \${1:-main}...HEAD`

1. code-reviewer 서브에이전트로 diff 리뷰 위임
2. Critical, Warning, Suggestion으로 정리
3. 마지막 줄에 머지 가능 여부 판정

수집 + 위임

명령의 골격

Ch.2 결합

reviewer 재사용

판정 규칙

자동화 접점

인자 기본값

\${1:-main} 관용구

실전 명령 2 / 배포 점검

disable-model-invocation의 용례

.claude/commands/deploy.md

```
---
description: 스테이징 배포 전 점검과 실행
argument-hint: "<환경: staging|prod>"
allowed-tools: Bash(git *), Bash(npm run *), Read
disable-model-invocation: true
---
```

```
대상 환경: $1
테스트: !`npm run test 2>&1 | tail -3`
미커밋: !`git status --short`
```

체크리스트 통과 시에만 배포 명령을 제시하고, 실패 항목이 있으면 배포를 중단하라.

invocation 금지

사용자 전용

관문 로직

통과 시에만 제시

도구 제한

배포 명령 미포함

ask 병행

권한 이중 관문

팀 라이브러리

명령 자산의 성장 경로

TEAM LIBRARY

개인 명령에서 출발해 프로젝트 커밋으로, 성숙하면 플러그인으로.

명령 자산도 코드처럼 리뷰와 버전을 갖습니다.

SEED

개인 검증

~/.claude/commands에서
시작, 반복 확인.

SHARE

프로젝트 승격

.claude/commands 커밋,
PR 리뷰 대상화.

SCALE

플러그인화

다저장소 공통은 플러그인 +
마켓 배포.

CURATE

큐레이션

명명 규칙, 설명 품질, 분기
정리 담당자.

3단 성장

개인, 팀, 조직

PR 리뷰

명령도 코드

네임스페이스

폴더 규칙

Ch.3 통제

마켓 allowlist

선택 가이드

Command vs Skill vs Subagent

짧은 정형 프롬프트

Command 한 장

요약, 번역, 템플릿

절차 + 동봉 자산

Skill 폴더

체크리스트, 스크립트 묶음

무거운 실행 격리

Skill context: fork

본진 컨텍스트 보호

역할 + 도구 제한 워커

Subagent (Ch.2)

리뷰어, 스캐너

자동 반응

Hook (Part 3, 4)

이벤트 구동은 흑으로

판별 축

누가 부르나, 상태 격리 필요한가

이 두 질문이면 충분

Part 7 Recap

Commands와 Skills 핵심 정리

KEY TAKEAWAY

반복이 보이면 명령으로 굳히고, 자산이 붙으면 스킬로 승격하며, 팀 가치가 생기면 커밋과 플러그인으로 유통합니다.

핵심 정리

- 파일명 = /명령, 폴더 = 네임스페이스
- \$1, \$ARGUMENTS + argument-hint
- !실행과 @참조로 컨텍스트 수집
- 스킬: 폴더, fork, once 혹
- 성장: 개인, 프로젝트, 플러그인

다음 파트

- Part 8 통합과 트러블슈팅
- .claude 폴스택 구성
- 진단 4도구
- 설정이 안 먹을 때
- 보안 체크리스트

CHAPTER 04 / PART 08

통합과 트러블슈팅

한 저장소에 다 모으고, 안 되면 가르기

- .claude 폴스택 통합 예시
- 진단 4도구: /context /doctor /hooks /mcp
- 설정이 안 먹을 때의 진단 흐름
- 보안 체크리스트

통합 예시 / .claude 풀스택

챕터 4가 커밋된 저장소의 모습

settings.json

permissions 팀 표준, env 관측, hooks 3종

hooks/

format.sh, block-rm.sh, mcp-write-guard.sh

commands/, skills/

pr-review, deploy + deploy-check 스크립트

.mcp.json

corp-wiki, github, db 팀 서버 3종

agents/

code-reviewer 등 Chapter 2 워커들

진단 4도구

무엇이 실제로 로드되었나 (debug-your-config)

/context

컨텍스트 점유 분해: 지침, 도구, 대화

비대 원인 특정

/doctor (claude doctor)

환경, 설정 검증과 자동 수정 제안

무효 항목 출처 표시

/hooks

혹 브라우저: 타입, 매치, 소스

Part 4 복습

/mcp

서버 상태, 도구, 인증

Part 5 복습

보조

/status, /permissions, --verbose

구성 스냅샷과 로그

설정이 안 먹을 때

우선순위와 스코프의 이분 진단

STEP 1

로드 확인

/doctor로 파일 유효성, 오류

STEP 2

출처 확인

/status, /permissions
스코프 표기

STEP 3

우선순위

상위 스코프의 동일 키 덮음
여부

STEP 4

예외 키

model, outputStyle은
재시작 필요

그래도 안 되면: 새 터미널 + --bare로 최소 기동 후 하나씩 복원

doctor 먼저

유효성 관문

출처 표기

스코프 특징

덮임 확인

우선순위 함정

--bare

최후 이분법

Hooks 트러블슈팅

혹이 안 돌거나 잘못 돌 때

안 발화

매처 불일치, 이벤트 오선택

/hooks로 등록, 매처 확인

mcp__server 무매칭

정확 문자열로 해석됨

__.* 접미 추가

차단이 안 됨

http 비2xx는 비차단

2xx + deny 본문으로

스크립트 오류

경로, 실행 권한, jq 부재

단독 실행으로 재현

프롬프트 지연

UserPromptSubmit 무거움

async 또는 30초 내로

전부 침묵

disableAllHooks 잔존

설정 스코프 전수 확인

MCP 트러블슈팅 요약

Part 6 진단표의 압축판

안 보임

스코프, 신뢰 승인

/mcp 출처 확인

연결 실패

명령, URL, 네트워크

단독 실행, curl

인증 오류

OAuth 만료, 변수 미설정

재인증, env

호출 거부

권한 vs 조직 통제 구분

/permissions, 관리자

느림

서버 과다

스코프 축소, 도구 검색

보안 체크리스트

커밋 전 마지막 점검

SECURITY GATE

.claude 폴스택은 강력한 만큼 저장소발 실행 경로이기도 합니다.
커밋 전 이 네 칸을 점검하는 습관이 팀을 지킵니다.

GATE 1

비밀 없음

settings, .mcp.json,
스크립트에 토큰 실값 금지.

GATE 2

deny 기본선

.env*, secrets/**,
파괴 명령 차단 유지.

GATE 3

혹 검토

혹 스크립트도 코드 리뷰,
외부 전송은 사유 명시.

GATE 4

서버 신뢰

.mcp.json 서버 출처 확인,
쓰기 도구 관문.

0 secrets

1번 관문

기본선 유지

deny 3종

혹도 코드

리뷰 대상

출처 확인

서버 채택 기준

Part 8 Recap

통합과 트러블슈팅 핵심 정리

KEY TAKEAWAY

클론 즉시 팀 표준이 서는 저장소가 목표입니다.

안 되면 doctor로 유효성, 출처 표기로 스코프, --bare로 이분하는 흐름을 몸에 익히세요.

핵심 정리

- 풀스택: settings, hooks, commands, mcp, agents
- 진단 4도구 + status, verbose
- 범인 1순위: 상위 스코프 덮음
- http 혹은 비차단 의미론 주의
- 커밋 전 보안 4관문

다음 파트

- Part 9 Recap & Labs
- 챕터 핵심 요약
- FAQ
- 실습 랩 4종
- 다음 챕터 예고

CHAPTER 04 / PART 09

Recap & Labs

챕터 정리와 실습 4종

- Chapter 4 핵심 요약과 체크리스트
- FAQ
- 실습 랩 4종
- 학습 자료와 다음 챕터 예고

Chapter 4 핵심 요약

여덟 파트를 여섯 문장으로

SETTINGS

체계

4스코프 5단 우선순위, 권한은 병합, 저장 즉시 리로드.

PERMISSIONS

권한

Tool(specifier) 한 형식, deny 우선, 모드 6종.

HOOKS

훅

30 이벤트 3 케이던스, 5 핸들러, 침묵은 무결정.

MCP

연결

3 프리미티브 4 전송 3 스코프, 도구 검색으로 규모 대응.

COMMANDS

명령

Markdown이 /명령, 스킬로 승격, 플러그인 유통.

OPS

운영

진단 4도구, 3중 방어, 커밋 전 보안 4관문.

학습 목표 체크리스트

네 가지 목표 자가 점검

SELF CHECK

네 문장에 답할 수 있으면 목표 달성입니다.

막히면 표기된 파트로 복습하세요.

CHECK 1

설정 설계

스코프와 우선순위, 리로드
예외 설명. (Part 1)

CHECK 2

권한 조율

규칙 문법과 모드로 팀 표준
작성. (Part 2)

CHECK 3

혹 자동화

이벤트, 매처, 결정 출력으로
레시피 구현. (Part 3, 4)

CHECK 4

확장 통합

MCP와 명령을 커밋해 폴스
택 구성. (Part 5-8)

4 Checks

목표 점검

Part 참조

복습 경로

Labs 4종

미완은 실습으로

커밋 기준

구축 판정

FAQ / 자주 나오는 질문

현장 질문 여섯 가지

설정을 고쳤는데 반영이 안 돼요

model, outputStyle만 재시작

그 외는 덮임 의심

allow에 넣었는데 계속 물어봐요

상위 deny 또는 ask 병합 확인

/permissions 출처

혹이 승인해 줄 수 있나요

침묵은 무결정, allow 결정도 가능

기본은 deny 용도

mcp__서버 매치가 안 걸려요

정확 문자열 해석 함정

__.* 접미 필수

서버가 많아 시작이 느려요

도구 검색 + 스코프 축소

Part 5, 6

명령과 스킬 차이가 뭔가요

한 장이면 명령, 자산 동봉이면 스킬

Part 7 가이드

Lab 1 / settings와 permissions

소요 20분

LAB 1 STEPS

```
cd claude-code-workshop/ch4/lab1 && claude
```

1. 팀 표준 작성 (제공 골격 완성)

> .claude/settings.json에 \$schema와 permissions를 채워줘. lint, test allow / push ask / .env* deny

2. 우선순위 체험

> settings.local.json에 같은 키를 다른 값으로 넣고 어느 쪽이 이기는지 /permissions로 확인해줘

3. 리로드 체험: 파일 수정 후 무재시작 반영 확인

20분

예상 소요

\$schema

자동완성 체험

local > project

우선순위 확인

위치

무재시작 반영

Lab 2 / Hooks 포맷터와 가드

소요 25분

LAB 2 STEPS

1. 포맷터 혹은 등록 (제공 format.sh 사용)

> PostToolUse Edit|Write에 format.sh를 exec form으로 등록해줘. 파일 수정 후 자동 포맷 확인.

2. 가드 혹은

> PreToolUse Bash + if "Bash(rm *)"로 block-rm.sh 등록

> /tmp/x 를 rm -rf 로 지워줘 # deny 사유 확인

3. /hooks에서 소스 라벨 확인

4. disableAllHooks로 전체 정지 후 복원

25분

예상 소요

exec form

args 등록 체험

deny JSON

차단 확인

이분 탐색

전체 정지 체험

Lab 3 / MCP 추가와 도구 검색

소요 25분

● ● ● LAB 3 STEPS

1. stdio 서버 추가

\$ `claude mcp add playwright -- npx -y @playwright/mcp@latest`

2. 프로젝트 스코프 공유

\$ `claude mcp add -s project github --transport http \`
`https://api.githubcopilot.com/mcp/`

.mcp.json 생성 확인, /mcp에서 인증

3. 사용: 도구 자연어 호출 + 권한 allow 추가

4. /context로 MCP tools 점유 확인,

서버 추가 전후 비교

25분

예상 소요

2 전송

stdio + http

OAuth

/mcp 인증 체험

/context

예산 관측

Lab 4 / 커스텀 명령

소요 20분

● ● ● LAB 4 STEPS

1. 첫 명령

- > .claude/commands/standup.md 만들어줘.
어제 커밋(!git log)과 미커밋 상태를 요약하는 명령.
- > /standup

2. 인자와 힌트

- > fix-issue.md를 \$1, \$2와 argument-hint로 작성해줘

3. 스킬 승격

- > standup을 skills/standup/으로 승격하고
체크리스트 리소스를 동봉해줘
- # /hooks, 목록에서 등장 확인

20분

예상 소요

!실행

컨텍스트 수집

\$1 인자

위치 인자 체험

스킬 승격

성장 경로 체험

추가 학습 자료

이 챕터 다음에 읽을 것들

Settings

code.claude.com/docs/ko/settings

전 키 사전

Permissions

docs 내 permissions, permission-modes

규칙과 모드 원전

Hooks

docs 내 hooks(레퍼런스), hooks-guide

30 이벤트 전 스키마

MCP

docs 내 mcp, mcp-quickstart, managed-mcp

구성과 통제

Skills

docs 내 skills, features-overview

명령, 스킬, 선택 지형

워크샵 코드

github.com/whchoi98/claude-code-workshop

본 과정 실습 자료

Next / Chapter 5 예고

CLI Reference와 자동화

CHAPTER 5 PREVIEW

대화형을 넘어 스크립트와 파이프라인의 세계입니다.
헤드리스 실행과 플래그 전집, 세션 제어, CI 통합을 다룹니다.

NEXT 1

플래그 전집

-p, --output-format, --
resume 등 CLI 완전 정복.

NEXT 2

헤드리스

파이프라인 속 Claude,
JSON 출력 파싱.

NEXT 3

세션 제어

continue, resume, fork
와 전송.

NEXT 4

자동화 통합

CI, cron, Routines와의 결
합.

Ch.5

CLI Reference

Day 3

워크샵 일정

자동화 관점

시선 전환

To be continued

다음 세션에서

References

본 챕터 공식 문서 출처 (code.claude.com)

Settings	docs/ko/settings	Part 1 전반
Permissions	docs/ko/permissions , permission-modes	Part 2
Hooks	docs/ko/hooks , hooks-guide	Part 3, 4
MCP	docs/ko/mcp , mcp-quickstart , managed-mcp	Part 5, 6
Skills, Commands	docs/ko/skills , features-overview	Part 7
Debug	docs/ko/debug-your-config	Part 8

Thank you!

Choi, WooHyung / Prin. Solutions Architect, AWS Korea

whchoi@amazon.com | linkedin.com/in/woohyungchoi

Workshop code: github.com/whchoi98/claude-code-workshop